LECTURE 5 · A STORY IN FOUR CHAPTERS

Real-Time Streaming, AI-Powered UI & Agentic Interaction Patterns

Today we design and build OrderMitra together — a chat assistant living inside Session 4's Orders page. We start with the design, and every concept shows up because the design needs it.

Instructor
Dept. of Computer Science and Engineering, IIT Kharagpur
mainack@cse.iitkgp.ac.in

Today's Story, in Four Chapters

Ch.1	Server-Sent Events — one-way LLM streaming
Ch.2	Vercel AI SDK — streaming, loading states, confidence indicators
Ch.3	Agentic interaction patterns — tool calls, multi-step reasoning
Ch.4	Human-in-the-loop UI — confirmation, override, abort/retry

By the final page, you'll be able to...

Build OrderMitra end to end: a real-time interface that streams LLM responses, calls tools to search and act on real order data, and always asks before doing anything destructive.

RECAP

The Story So Far

Four sessions in — here's the thread this one picks up.

- Session 1 — AI pair-programming discipline: specify, verify, override
- Session 2 — Next.js App Router, Server Components, strict TypeScript
- Session 3 — accessible components, verify what AI writes before it ships
- Session 4 — Prisma, OAuth, TanStack Query, review AI-drafted schemas
- Today — the Orders page gets a voice, and hands — carefully

> *Same discipline as every session before it — verification now applies to what an AI model says and does in real time.*

CHAPTER 0

Let's Design OrderMitra, Before We Build Anything

Every concept today exists because this design needs it. Nothing is taught because it's interesting — only because OrderMitra can't work without it.

The Ask, As a Product Manager Would Actually Say It

This is the whole brief. Everything today is us turning this into working software.

- "Add a chat assistant to the Orders dashboard"
- "It should answer questions like a helpful teammate, not a search box"
- "It should be able to look up specific orders when asked"
- "It should never take a destructive action without asking first"

Four sentences. By the end of today, each one is a chapter, built and working.

Turning the Brief Into an Engineering Design

Before writing a line of code, a software engineer asks: what does each requirement actually demand of the system?

"Like a helpful teammate"

- Feels responsive — answers arrive as they're generated, not after a long wait
- This is a UX requirement first, a technical one second

"Look up specific orders"

- The assistant needs real data, not guesses
- It must be able to run real code — a database query — mid-conversation

"Never act without asking"

- Some of that "real code" changes data, not just reads it
- Those actions need a checkpoint a human controls

Three requirements, three technical demands — this table is the actual table of contents for today, just not labelled with jargon yet.

The System We're Building, Sketched Out

One request, five boxes — we'll fill in why each box exists as we go, chapter by chapter.

system design: OrderMitra



We haven't written a line of code yet — this is purely the design conversation. Chapter 1 starts with the leftmost problem: getting words from server to browser as they're generated.

Keep this picture in mind. Every "why are we talking about this?" moment today has an answer: because this box needed it.

CHAPTER 1

Making OrderMitra Feel Alive

Back to the design: the browser needs words to arrive as they're generated, not all at once. This chapter is entirely about solving that one box.

The Goal, Named Precisely

From the design conversation: "feels like a helpful teammate." Here's what that actually demands.

What We're Solving

- OrderMitra's answer should start appearing almost instantly
- It should keep arriving, word by word, while the model is still "thinking"
- The user should never stare at a frozen, blank screen

This entire chapter is one engineering problem: how do words travel from wherever the AI model runs to the user's browser, as they're generated — not after?

A Feeling You Already Know

Before any technical term — this is a UX pattern you've used a hundred times.

Think of WhatsApp

- You've seen the "..." typing indicator, then the message arriving
- You know instantly that the other person is composing a reply, live
- You never wonder "is this app broken?" — the feedback is continuous

That's the exact feeling we want from OrderMitra — except instead of three dots, we show the actual words as the model generates them.

That Feeling Has a Name: Streaming

Our first formal term — now that we already know the feeling it's named after.

In Plain Words

- Streaming means sending a response in small pieces, as they become ready
- Instead of waiting for the whole thing, then sending it all at once
- The WhatsApp typing indicator is one example. OrderMitra's answer is another.

Streaming isn't an AI-specific idea — it's a general pattern for "show progress as it happens." AI chat is simply where we're applying it today.

Quick Refresher: How Does an LLM Actually Answer?

This is the piece that makes streaming not just possible, but natural, for a model's output specifically.

In Plain Words

- A model doesn't compose the full answer internally, then reveal it
- It generates one word (technically, one "token") at a time
- Each new word is predicted based on everything generated so far
- This is exactly why ChatGPT, Claude, and OrderMitra all "type" their answers

The model is already producing words one at a time internally — streaming just means we stop hiding that and show each word the moment it's ready.

So Let's See What Happens If We Don't Do This

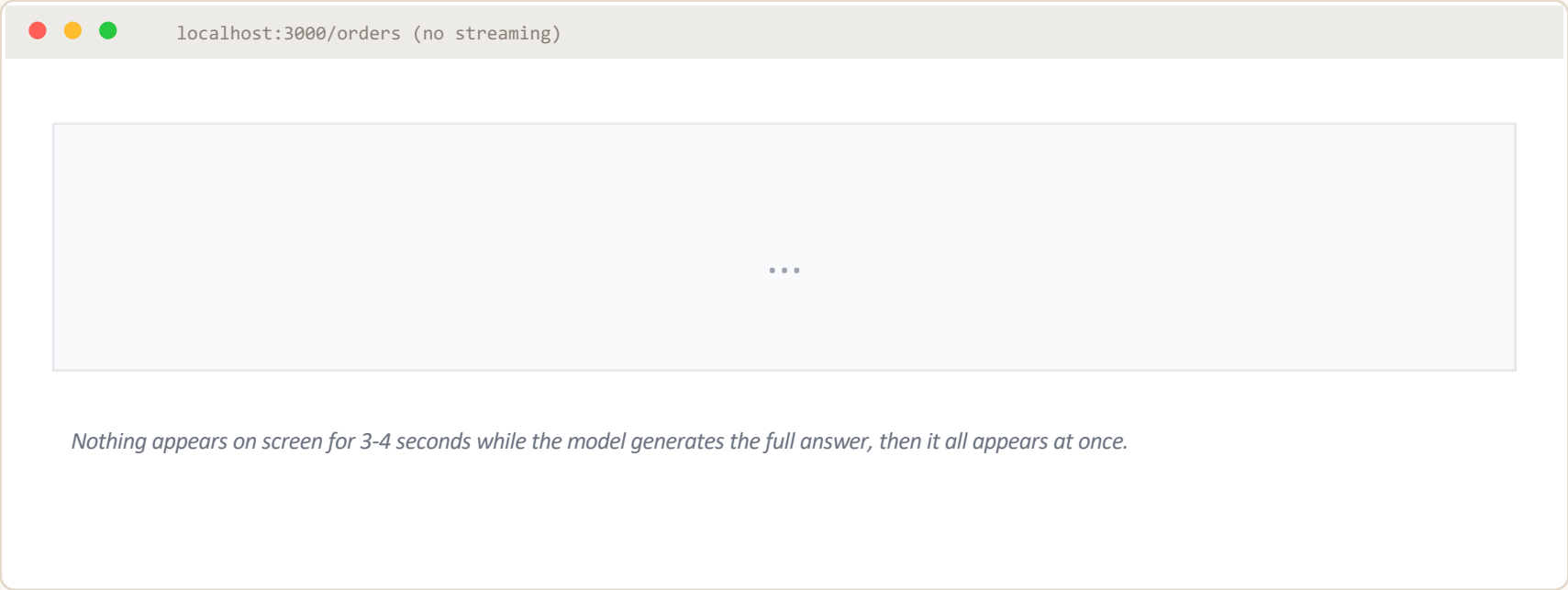
Before the fix, the actual failure — this is the version we're about to replace.

- OrderMitra's server asks the model for a full answer
- The model generates it, word by word, exactly as we just described
- But we wait — and only send it to the browser once every word is ready
- The user sees nothing for 3-4 seconds, then everything appears at once

We know now why this feels broken — we're throwing away the word-by-word generation we just learned about, and gluing it back into one lump.

The Old Way, on Screen

Exactly the failure from the last slide, made visible.



Technically correct, and it still feels broken — this is precisely the "is this app broken?" moment WhatsApp's typing indicator was built to prevent.

The Real Engineering Question

Now we're ready for the actual problem this chapter solves.

- We know the model generates words one at a time
- We know we want the browser to show each word the moment it's ready
- The model runs on a server, far from the user's browser
- So: how does a server actually send data to a browser, piece by piece, live?

Everything from here — polling, persistent connections, SSE, EventSource — is us answering this one question, one idea at a time.

First Instinct: Just Ask Again and Again

A completely reasonable first idea — worth trying on paper before we rule it out.

The Idea ("Polling")

- The browser asks the server "anything new?" every second
- The server replies, even if the answer is "no, nothing yet"
- Repeat until the full answer has arrived

Why It Breaks for OrderMitra

- Most of those requests come back with nothing new — wasted effort
- A 50-word answer means dozens of separate round trips
- At real usage, every open chat window hammering the server every second adds up fast

> *Polling isn't wrong, exactly — it's just asking over and over, when what we actually want is for the server to speak up the moment it has something to say.*

What We Actually Need: A Connection That Stays Open

The insight that fixes polling's waste.

In Plain Words

- Normally, a browser asks once, gets one reply, and the connection closes
- Instead: open a connection and deliberately leave it open
- The server can now send more data down that same connection, any time it wants — no re-asking required

> *This single idea — a connection kept open on purpose — is the actual mechanism behind everything we're about to build.*

This Idea Already Has a Name and a History

Server-Sent Events (SSE) — a set of rules both browser and server agree to follow, so the "stay open and push data" idea actually works reliably.

- 2004 — the idea is first conceptualized: let a server push updates
- 2006 — standardized as part of HTML5 by Ian Hickson
- Built originally for stock tickers, news feeds, live scores — one-way updates
- 2023 onward — turns out to be the exact right shape for LLM tokens
- Today — the transport underneath nearly every streaming chat UI, including OrderMitra's

SSE wasn't built for AI at all — it already solved our exact problem, 17 years before OrderMitra needed it.

Where Else You've Already Seen This

A quick aside — SSE is not an OrderMitra-only trick.

You've Used This Before

- Live cricket score updates on a scoring app
- A stock ticker updating without refreshing the page
- GitHub Actions logs scrolling live as a build runs
- This very chat interface, streaming Claude's own replies to you

Once you notice the pattern — "this is updating without me refreshing" — you'll spot SSE (or something like it) everywhere.

Where SSE Fits, Among the Alternatives

Three ways a browser can get "new" information — only one is shaped like an LLM's output.

Polling

- Client repeatedly asks "anything new?"
- Wasteful, adds latency between updates

SSE

- Server pushes tokens as they're generated
- Plain HTTP — no special protocol upgrade
- One-way only: server to client

WebSockets

- Full bidirectional channel
- More setup, more infrastructure
- Overkill for one-way token streaming

OrderMitra's data only flows one direction — server to browser — so SSE is the right-sized tool, not a workaround.

WebSockets, With a Real AI-Native Example

Worth actually understanding, not just ruling out — you'll reach for this on a different feature.

What Makes It Different

- A WebSocket is a channel BOTH sides can send on, any time
- SSE: only the server speaks. WebSockets: either side can
- More setup — a persistent bidirectional connection to manage, not just a one-way pipe

An AI-Native Example That Needs It

- A live voice conversation with an AI agent — you're speaking while it's replying
- A multiplayer AI pair-programming session, where two people's edits both stream in
- OrderMitra never needs this — it only ever pushes answers outward

The rule isn't "SSE good, WebSockets bad" — it's "which direction does data actually need to flow?" For OrderMitra, one direction is genuinely all there is.

One More Piece: Who Listens on the Browser Side?

SSE handles the server's half. We still need something on OrderMitra's frontend to actually receive it.

In Plain Words

- EventSource is a built-in browser feature — no library needed
- You give it a URL; it opens the persistent connection for you
- Every time the server sends a piece of data, EventSource hands it to your code
- It even reconnects automatically if the connection drops

You rarely touch EventSource directly for LLM chat — Chapter 2's SDK wraps it — but knowing it exists demystifies everything that comes next.

Quick Reminder: Route and Component

Two words from Session 2 we're about to lean on heavily — worth a 10-second refresh.

A "Route"

- A file that handles a server-side request — lives under app/api/
- app/api/stream/route.ts answers requests to /api/stream
- This is where OrderMitra's server-side code runs

A "Component"

- A piece of UI — a function that returns what the browser shows
- Marked "use client" when it needs interactivity in the browser
- This is where OrderMitra's chat window actually renders

Everything in this chapter is just: what does the route send, and what does the component do with it?

What Is a ReadableStream?

The building block the server-side code is written on top of — this one runs on the server itself, not the browser.

In Plain Words

- A ReadableStream is a built-in JavaScript object for producing data piece by piece
- You give it a start() function that runs once, when someone starts reading
- Inside it, you call controller.enqueue() each time you have a new piece to send
- controller.close() tells it there's nothing more coming

This isn't an SSE-specific tool — it's a general "produce data over time" primitive. SSE is just one particular way of using it.

What the Server Side Actually Looks Like

This is the whole mechanism from this chapter, in code — understanding it demystifies Chapter 2's streamText().

app/api/stream/route.ts

```
export async function GET() {
  const stream = new ReadableStream({
    async start(controller) {
      for (const chunk of ["Hello", " ", "world"]) {
        controller.enqueue(`data: ${chunk}\n\n`);
        await new Promise((r) => setTimeout(r, 200));
      }
      controller.close();
    },
  });
  return new Response(stream, {
    headers: { "Content-Type": "text/event-stream" },
  });
}
```

What This Code Does

GET() is the route's handler • new ReadableStream(...) opens the connection Chapter 1's earlier slide described • the for loop sends "Hello", " ", "world" one at a time, 200ms apart, each formatted as an SSE data: line • controller.close() ends the stream once done

AI Helps Here

Prompt: "Write a minimal Next.js route handler that streams the words 'Hello world' using the SSE ReadableStream pattern, one word every 200ms."

You Verify This

Confirm Content-Type is exactly text/event-stream — this is the one line that tells the browser "treat this as SSE."

Three words, three chunks, one open connection — the entire mechanism this chapter has been building toward.

What the Browser Side Actually Looks Like

EventSource, doing exactly what its name says — the other half of the last slide.

app/stream-demo.tsx

```tsx
"use client";
import { useEffect, useState } from "react";

function StreamDemo() {
  const [text, setText] = useState("");
  useEffect(() => {
    const source = new EventSource("/api/stream");
    source.onmessage = (event) => setText((t) => t + event.data);
    return () => source.close();
  }, []);
  return <p>{text}</p>;
}
```

What This Code Does

"use client" marks this as a Component that runs in the browser • new EventSource(...) opens a connection to our route and starts listening • onmessage fires once per chunk that arrives, appending it to existing text • the cleanup function closes the connection if the component unmounts

AI Helps Here

Prompt: "Write a client component that connects to /api/stream with EventSource and appends each incoming chunk to a paragraph on screen."

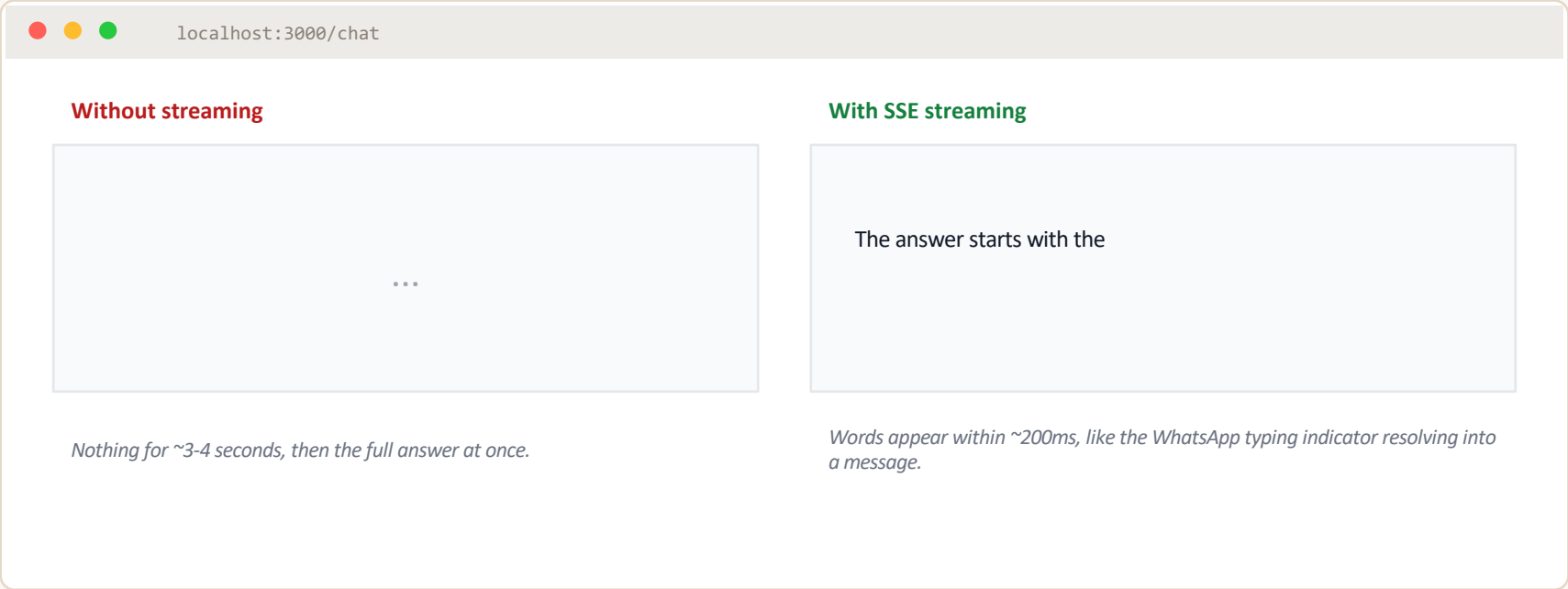
You Verify This

onmessage fires once per chunk — confirm you're appending to existing text, not replacing it, or you'll only ever see the last word.

This exact pair of files — one server, one browser — is the whole chapter, distilled.

The Payoff, Side by Side

The same response, two different waits — this is what those two files buy us.



Same total generation time — the only thing that changed is whether the user is left wondering "is this still working?"

Your Thumb Rule: SSE vs WebSockets vs Polling

The chart to actually keep — the one you'll reach for on your next real project, not just today's example.

? Streaming one-directional LLM tokens?	→ **SSE**
? Client also needs to send real-time messages back?	→ **WebSockets**
? Infrequent updates, simplicity over latency?	→ **Polling**
? Need automatic reconnection with minimal code?	→ **SSE — built into EventSource**

🔨 HANDS-ON STARTING

Feel a Real Stream Arrive

~10 min, led by your TA

Your TA will walk you through building a tiny SSE counter — the exact mechanism from this chapter, in your own hands, before OrderMitra exists.



HANDS-ON COMPLETE

Streaming, Demystified!

You just watched real data arrive one piece at a time. Back to the lecture — this exact mechanism is what Chapter 2 wraps in a much friendlier API.

Chapter 1 Key Terms

Background material worth keeping.

Terms

- Streaming — sending a response in small pieces, as they're ready
- Persistent connection — a connection deliberately kept open for more data
- SSE (Server-Sent Events) — a one-way streaming protocol built on HTTP
- EventSource — the browser API that consumes an SSE stream
- Token — one unit (roughly a word or word-piece) of a model's generated output

END OF CHAPTER 1

Making OrderMitra Feel Alive

- The goal was a feeling — like WhatsApp's typing indicator — before it was any technology
- LLMs generate one token at a time, which is exactly why streaming feels natural for them
- Polling asks over and over; SSE keeps a connection open and lets the server speak up
- SSE (2004/2006) and EventSource are the two-sided mechanism — server sends, browser listens
- Every idea in this chapter existed because the design in the first ten minutes demanded it

The takeaway: *Understanding the protocol underneath makes every streaming bug in Chapter 2 far less mysterious.*

Today's Story, in Four Chapters



~~Server-Sent Events — one-way LLM streaming~~

Ch.2 Vercel AI SDK — streaming, loading states, confidence indicators

Ch.3 Agentic interaction patterns — tool calls, multi-step reasoning

Ch.4 Human-in-the-loop UI — confirmation, override, abort/retry

Up next

OrderMitra's actual brain — where the words come from, and why nobody hand-writes this plumbing from scratch anymore.

CHAPTER 2

Where OrderMitra's Words Actually Come From

Chapter 1 solved HOW words travel to the browser. This chapter solves WHAT generates them, and how we build the real feature without hand-rolling every piece.

Could We Just Write More Raw SSE Code?

A fair question, given Chapter 1 already worked.

- Yes — technically, Chapter 1's ReadableStream pattern could send a real model's output too
- But every AI product needs the same plumbing: call a model, stream its tokens, track loading state, handle errors
- Writing all of that by hand, correctly, every single time, is exactly the kind of repetitive work engineers eventually package into a tool
- This chapter is about the tool the industry converged on for exactly this job

> *This is a real engineering decision — hand-roll infrastructure everyone rebuilds anyway, or use a toolkit built by people who've already solved it. We're choosing the toolkit.*

How the Vercel AI SDK Came to Be

Not a random company — this is who actually noticed the repetition and built the fix.

- Early 2023 — Scale, Jasper, Perplexity, Runway and others launch on Next.js
- Every one of them hand-builds the same streaming chat plumbing
- Vercel (who makes Next.js) notices the exact pattern repeating across unrelated companies
- June 2023 — the Vercel AI SDK ships: npm i ai
- Built by the same team behind Next.js — provider-agnostic from day one

> *The SDK didn't invent streaming chat — it packaged a pattern Vercel watched dozens of teams reinvent independently, which is exactly the repetition we just described.*

So What Does "Using an SDK" Actually Mean?

Concretely — not as an abstract idea.

Without the SDK (Chapter 1's Way)

- You write the ReadableStream by hand
- You manually format each chunk as an SSE message
- You track loading/error state yourself, on every feature

With the SDK

- You call one function: streamText()
- It handles the ReadableStream, the SSE formatting, and more, internally
- One hook (useChat) replaces all the manual client-side bookkeeping too

> *"Using an SDK" just means: someone already wrote and tested the repetitive plumbing, and packaged it as a function you call instead of code you maintain.*

The Goal for This Section, Stated Up Front

Top-down: here's exactly what we're about to build, before we see any code for it.

One Route

- Receives OrderMitra's conversation so far
- Sends it to a model provider
- Streams the reply back using streamText()

One Component

- Calls useChat() to talk to that route
- Shows the reply as it streams in
- Shows a loading state while waiting

Two pieces. Everything below is just filling in these two boxes.

What Is a "Model Provider"?

One term before the first piece — streamText() needs to be told which model to actually call.

In Plain Words

- A model provider is the company running the actual AI model
- Anthropic (Claude), OpenAI (GPT), Google (Gemini) are all model providers
- You send them a request; they send back generated text
- The SDK can talk to any of them through the same code shape

Not to be confused with a React "Provider" from Session 4 — same word, unrelated meaning. Context tells you which one is meant.

Piece One: The Streaming Route

app/api/ordermitra/route.ts

```
import { streamText } from "ai";
import { anthropic } from "@ai-sdk/anthropic";

export async function POST(req: Request) {
  const { messages } = await req.json();
  const result = streamText({
    model: anthropic("claude-sonnet-4-6"),
    messages,
    system: "You are OrderMitra, an Orders assistant.",
  });
  return result.toDataStreamResponse();
}
```

What This Code Does

POST() receives the conversation • streamText() replaces Chapter 1's ReadableStream by hand • model picks the provider • system sets OrderMitra's persona • toDataStreamResponse() returns it as a stream

AI Helps Here

Prompt: "Create app/api/ordermitra/route.ts using streamText from the ai package with the Anthropic provider, model claude-sonnet-4-6, and a system prompt describing OrderMitra."

You Verify This

streamText() handles the SSE plumbing from Chapter 1 for you — this is the entire server side of a streaming chat.

What Is a "Hook," Again?

A quick recap in case this is your first deck this series — useChat is one of these.

In Plain Words

- A hook is a special function whose name starts with "use"
- It lets a component "hook into" a feature React manages for you
- useChat is a hook built specifically for talking to a streaming chat route

> *Every time you see use-something, expect it to hand you state and behavior you'd otherwise have to wire up by hand.*

Piece Two: The Chat Panel

app/orders/ordermitra-panel.tsx

```tsx
function OrderMitraPanel() {
  const {
    messages, input, handleInputChange, handleSubmit, status,
  } = useChat({ api: "/api/ordermitra" });

  return (
    <div>
      {messages.map((m) => (
        <MessageBubble key={m.id} message={m} />
      ))}
      {status === "streaming" && <TypingIndicator />}
      <form onSubmit={handleSubmit}>
        <input value={input} onChange={handleInputChange} />
      </form>
    </div>
  );
}
```

What This Code Does

useChat() connects to our route and returns everything needed • messages is the conversation, rendered via MessageBubble • status drives the typing indicator • handleSubmit sends the input

AI Helps Here

Prompt: "Create OrderMitraPanel using useChat pointed at /api/ordermitra. Show a typing indicator while status is 'streaming'."

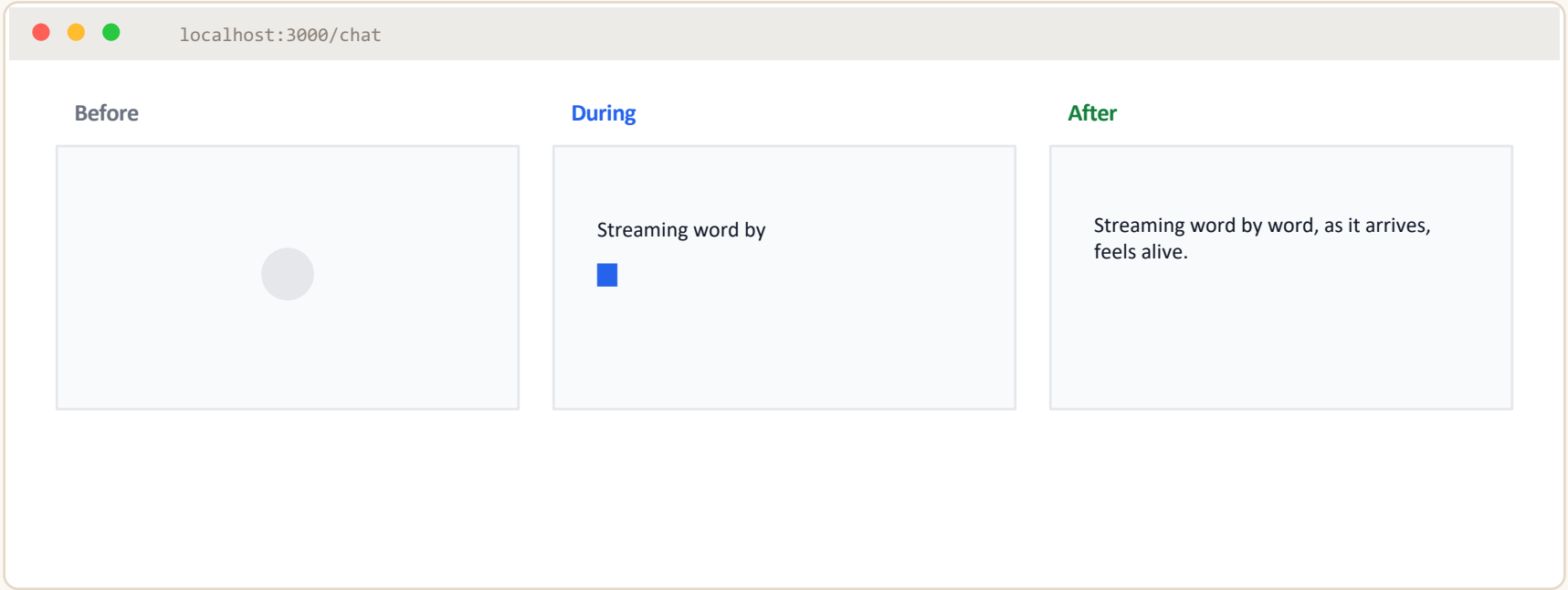
You Verify This

useChat owns message state, streaming updates, and request status — no manual EventSource wiring needed on the client either.

This is genuinely the whole feature — two files, and OrderMitra can already hold a streaming conversation.

The Three Loading States, Seeing status Directly

What the two pieces above actually put on screen.



"Loading state" just means: what does the screen show while we're waiting? status gives you three clean answers instead of guessing.

A RELATABLE MOMENT

"Is the app frozen, or is it just thinking?" That's the exact question status === "streaming" answers for the user — same feeling as the WhatsApp typing dots from Chapter 1.

One field, three states, and the user never has to wonder again.

OrderMitra Can Talk Now. But There's a Deeper Problem.

A callback to Session 1, now with a concrete OrderMitra scenario attached.

- Session 1: LLMs hallucinate, and sound equally confident either way
- Concrete scenario: OrderMitra says "Mr. Iyer's order shipped yesterday"
- Right now, the user can't tell if OrderMitra actually knows this, or is guessing
- A confident-sounding wrong answer is more dangerous than an uncertain one

> *We need OrderMitra to show its uncertainty, not hide it — that's the actual need behind everything in the rest of this chapter.*

How Do We Make a Model "Show Its Work"?

A fun way to think about it before the technical name.

Normally, Asking a Model a Question Is Like...

- ...asking an open-ended interview question
- It can answer however it wants, in whatever shape
- Great for conversation, bad for extracting a specific fact like "how sure are you?"

What We Actually Want Is More Like...

- ...handing it a form to fill in, not a blank page
- "Answer here. Confidence here. Did you find a source? Tick yes or no."
- Now the response is data we can actually build a UI around

This "hand it a form instead of a blank page" idea has a real name: structured output. We're about to build the form.

What Is "Structured Output," Formally?

The technical name for the "form, not a blank page" idea.

In Plain Words

- Normally an LLM just returns a blob of text
- Structured output means asking it to fill in a specific shape instead
- The response comes back as real, usable data — not text you have to parse by hand

Without structured output, you'd have to beg the model to "please format your confidence as a number" in the prompt and hope it listens. This makes it a guarantee, not a request.

What Is the "Form" Actually Made Of? Meet Zod

Same library from Session 4's Prisma schemas — but Zod itself is not a database. It just describes shapes.

In Plain Words

- Zod is a validation library — it is NOT a database, and it stores nothing
- It lets you describe the exact shape data should have, then checks data against that shape
- In Session 4, you used it to validate a database schema before a migration ran
- Here, it's literally the form: "answer is text, confidence is high/medium/low, groundedInSource is yes/no"
- Same tool, same instinct: describe the shape you expect, then enforce it

If Session 4's schema validation felt useful for catching bad database fields, this is the exact same safety net, now applied to a model's own output.

The Form, Written as a Zod Schema

app/api/answer/route.ts

```
const result = await generateObject({
  model: anthropic("claude-sonnet-4-6"),
  schema: z.object({
    answer: z.string(),
    confidence: z.enum(["high", "medium", "low"]),
    groundedInSource: z.boolean(),
  }),
  prompt: `Answer using only the provided context: ${context}`,
});
```

What This Code Does

generateObject() replaces generateText() when we need structured data back, not prose • schema is the Zod "form" from the last slide — three required fields, nothing else allowed • prompt is the actual question, same as always • result.object comes back already matching that exact shape

AI Helps Here

Prompt: "Use generateObject with a Zod schema to force the model to return answer, confidence (high/medium/low), and groundedInSource (boolean) as structured fields."

You Verify This

This is literally the "form" from two slides ago — three fields, forced every time, no exceptions.

The Same Form, Filled In for Order Summaries

Worth seeing twice — once you've written this pattern, every future confidence-scored feature reuses it.

app/api/order-summary/route.ts

```
const result = await generateObject({
  model: anthropic("claude-sonnet-4-6"),
  schema: z.object({
    summary: z.string(),
    confidence: z.enum(["high", "medium", "low"]),
    ordersReferenced: z.number(),
  }),
  prompt: `Summarize this week's orders: ${ordersData}`,
});
```

AI Helps Here

Prompt: "Reuse the confidence-scored generateObject pattern, this time summarizing a week of orders instead of answering a single question."

You Verify This

Notice the shape barely changed from the last slide — that reuse is the whole payoff of learning the pattern once.

Turning the Form Into a Badge on Screen

prompt to Claude Code

```
Add a confidence badge next to each AI answer: green for
high, amber for medium, red for low. Pull the value from
the confidence field returned by generateObject.
```

Pros

Correctly wired the badge color to the enum value, matched the existing design system's color tokens.

Cons

Used color alone to convey confidence — no text label. A colorblind user can't distinguish amber from red.

This is Session 3's "never rely on color alone" rule, resurfacing in a completely different chapter — the lessons compound.

What This Actually Looks Like in a Real UI

The same three states, all in one place — this is what a user actually sees, not just what the code returns.



Notice each badge also carries a text label, not just a color — the fix for the "Cons" on the last slide, applied.

Handling Streaming Errors & Interruptions

Failure Modes

- Network drop mid-stream
- Provider rate limit or 5xx mid-generation
- Client navigates away before completion

UI Response

- Show partial content — don't discard what arrived
- Offer a clear "Retry" action, not a silent failure
- status === "error" from useChat drives this state

A stream that fails halfway should never look identical to one that succeeded — always signal the truncation.

Aside: Where Else This Shows Up

OrderMitra is one example. Streaming and confidence indicators are everywhere once you're looking for them.

You've Likely Used This Already

- ChatGPT, Claude, Gemini — every major AI chat product streams
- GitHub Copilot's inline suggestions stream in as you type
- Spam filters and fraud detectors showing a "confidence score," not just yes/no
- Medical AI tools that flag a diagnosis as "high confidence" vs "needs review"

Once you've built one streaming, confidence-scored feature, you'll start recognizing the same two ingredients everywhere: a toolkit doing the plumbing, and a form forcing the model to show its uncertainty.

streamText vs generateText vs streamUI

Function	Output	Best For
generateText	Single string, awaited in full	Backend jobs, no streaming UI
streamText	Token stream to the client	Chat interfaces, long-form generation
generateObject	Single structured object	Confidence fields, classification
streamUI	Streamed React components	Generative UI — components, not just text

Default to streamText for chat; reach for generateObject whenever the model needs to self-report structured metadata.

Why Streaming Changes Perceived Performance

~200ms

typical time-to-first-token vs a multi-second full-response wait

3x

reported improvement in perceived responsiveness with streaming

1

hook (useChat) replaces manual fetch + state + SSE parsing

Users judge responsiveness by time-to-first-token, not time-to-completion — streaming changes the number that actually matters.

🔨 HANDS-ON STARTING

Build OrderMitra's Streaming Chat

~15 min, led by your TA

OrderMitra starts existing right now. Your TA will walk you through wiring up both pieces from this chapter, live.

HANDS-ON COMPLETE

OrderMitra Can Now Talk!

A real, working streaming chat, built by you this chapter. Back to the lecture — Chapter 3 gives it something to actually look up.

Chapter 2 Key Terms

Background material worth keeping.

Terms

- SDK — a pre-built toolkit for a specific job, here: AI-powered interfaces
- Model provider — the company running the AI model (Anthropic, OpenAI, etc.)
- streamText / useChat — the server and client halves of a streaming chat
- Structured output — forcing a model's response into a defined, validated shape
- Zod schema — the "form" that describes and enforces that shape

END OF CHAPTER 2

Where OrderMitra's Words Come From

- We chose a toolkit over hand-rolled plumbing — a real, deliberate engineering decision
- streamText() on the server and useChat() on the client cover most streaming chat needs
- Map the three loading states — before, during, after — onto useChat's status field
- Structured output (generateObject + Zod) turns a hallucination risk into a visible confidence badge
- OrderMitra can talk — a real, working streaming chat, built by you this chapter

The takeaway: *The SDK removes the plumbing — your job shifts to designing what the user sees while tokens are still arriving, and how honest the model is about what it doesn't know.*

Today's Story, in Four Chapters



~~Server-Sent Events — one-way LLM streaming~~



~~Vercel AI SDK — streaming, loading states, confidence indicators~~

Ch.3 Agentic interaction patterns — tool calls, multi-step reasoning

Ch.4 Human-in-the-loop UI — confirmation, override, abort/retry

Up next

OrderMitra can talk beautifully — and it's making things up. Chapter 3 gives it eyes into the real order data.

CHAPTER 3

Giving OrderMitra Real Eyes

OrderMitra can talk beautifully. Ask it about a real order, and right now, it just makes something up.

The Problem, Made Concrete

Let's actually watch OrderMitra fail, before fixing anything.

- You ask OrderMitra: "what's the status of Mr. Iyer's order?"
- OrderMitra has never seen the Orders database — it only knows language patterns
- It answers anyway, confidently: "Your order has shipped and is on its way!"
- This is invented. Not a lie exactly — just a plausible-sounding guess with no data behind it

> *Chapter 2's confidence badge would actually help here — but the deeper fix is giving OrderMitra a way to go check, instead of guessing at all.*

So, What's the Actual Fix?

Stated plainly, before any jargon.

The Idea

- Let the model run real code, mid-conversation, when it needs a real fact
- Not "trust the model's memory" — "let it go look"
- For OrderMitra: run an actual database query, then answer using the real result

This idea — letting a model call real functions — is what the rest of this chapter is about. The industry has a name for it: agentic behavior. We'll build up to that name properly.

What Does "Agentic" Actually Mean?

The word gets used loosely — here's the precise version, now that we know why we need it.

In Plain Words

- An "agent" is software that can decide its own next action
- "Agentic" describes an interaction where the model chooses, not just answers
- Right now, OrderMitra can only talk — it can't look anything up
- By the end of this chapter, it can decide to search, then decide what to do with the result

The model going from "only produces text" to "can choose to run code" is the entire meaning of "agentic" in this course.

How This Idea Got a Name and a History

- Oct 2022 — Yao et al. (Princeton + Google) publish the ReAct paper
- "Reasoning + Acting": the model thinks, then acts, then observes the result
- 2023 — OpenAI and Google ship native function calling
- Function calling makes the same think-act-observe loop reliable in production
- By late 2023 — function calling largely replaces hand-written ReAct prompting

The idea came from a research paper; the production version came from providers building it directly into the model.

What Is a "Tool," in Plain Words?

Before the formal anatomy — the simplest possible explanation.

In Plain Words

- A tool is just a function you let the model call
- You write the actual code — searching a database, sending an email, anything
- You describe it to the model in plain English
- The model decides when it's relevant, and what arguments to pass

The model never runs your code directly — it asks for a specific tool with specific arguments, and your server decides whether and how to run it.

Anatomy of a Tool Definition

Required Parts

- name — how the model refers to it
- description — when the model should use it
- parameters — a Zod schema (from Chapter 2!) the model fills in
- execute — the actual function that runs

The description does more work than the code — it's the model's only signal for when to reach for this tool over another.

Walking Through the Code Before You See It

In words, so the code on the next slide isn't the first time you meet these ideas.

What searchOrders Will Do

- Take a customer name and/or status as input, both optional
- Describe itself so the model knows: "use me when asked about specific orders"
- Run a real Prisma query — the same kind from Session 4 — against the Order table
- Hand the results back to the model as plain data

Nothing here is new — it's a Zod schema (Chapter 2) wrapping a Prisma query (Session 4), with a description on top.

searchOrders, Written Out

Exactly the four steps from the last slide, now as real code.

lib/tools/search-orders.ts

```
export const searchOrders = tool({
  description:
    "Search orders by customer name or status. Use this " +
    "whenever the user asks about specific orders.",
  parameters: z.object({
    customerName: z.string().optional(),
    status: z.enum(["pending", "shipped", "delivered"])
      .optional(),
  }),
  execute: async ({ customerName, status }) => {
    return prisma.order.findMany({
      where: { customerName, status },
    });
  },
});
```

What This Code Does

tool() wraps a description, a Zod schema, and a function into one callable thing • description tells the model when to use this • parameters (Zod) is the exact shape of arguments required • execute is a plain Prisma query, same pattern as Session 4

AI Helps Here

Prompt: "Create lib/tools/search-orders.ts. It should query Prisma's order model by customerName and/or status, both optional, with a description telling the model when to use it."

You Verify This

Confirm the Zod schema field names actually match your Order model's real field names from Session 4 — a mismatch here fails silently.

The Multi-Step Reasoning Loop

This is the actual ReAct loop from the 2022 paper, still the mental model today.

1. Think — the model reasons about what's needed next
2. Act — the model calls a tool with chosen arguments
3. Observe — the tool's result is appended to the context
4. Repeat or Answer — the model decides: another step, or a final response

maxSteps in the SDK bounds how many times this loop can run — the same guardrail every agent framework since 2022 has needed.

What Is "Reasoning," for a Model?

A word that sounds grand but means something specific and checkable here.

In Plain Words

- "Reasoning" here just means: deciding what to do next, given what's known so far
- It's not thinking the way a person thinks — it's pattern-based next-step prediction
- "Should I search for this order, or do I already have enough to answer?"
- That single decision, repeated, is the entire "reasoning" this course means

Keep this grounded — "reasoning" is a useful word, not a claim about the model having a mind. Treat it as a decision procedure to guardrail, not a black box to trust blindly.

What Is a "Guardrail"?

One more term before the multi-step code — we'll use this word constantly from here on.

In Plain Words

- A guardrail is a limit you set to contain what could go wrong
- Not a fix for bad behavior — a boundary so bad behavior can't go far
- maxSteps is a guardrail: even a confused model can only loop so many times
- You'll meet several more guardrails in Chapter 4

Guardrails assume the model will sometimes be wrong — they're not optional polish, they're the whole point of building this responsibly.

Walking Through Multi-Step Orchestration

In words, before the code — this is just Chapter 2's route with two additions.

What Changes From Chapter 2's Route

- Add a tools object — here, just searchOrders for now
- Add maxSteps: a hard cap on how many times the loop can run
- Everything else — streamText, the model, the system prompt — stays identical

Agentic behavior isn't a rewrite. It's Chapter 2's exact route, with a list of tools and a guardrail bolted on.

OrderMitra's Route, With Tools Added

app/api/ordermitra/route.ts

```
const result = streamText({
  model: anthropic("claude-sonnet-4-6"),
  messages,
  tools: { searchOrders },
  maxSteps: 3,
  system: "You are OrderMitra. Use tools to check real data.",
});

return result.toDataStreamResponse();
```

What This Code Does

Same route as Chapter 2, two new lines added • tools makes searchOrders available to the model • maxSteps: 3 bounds the think-act-observe loop from running away

AI Helps Here

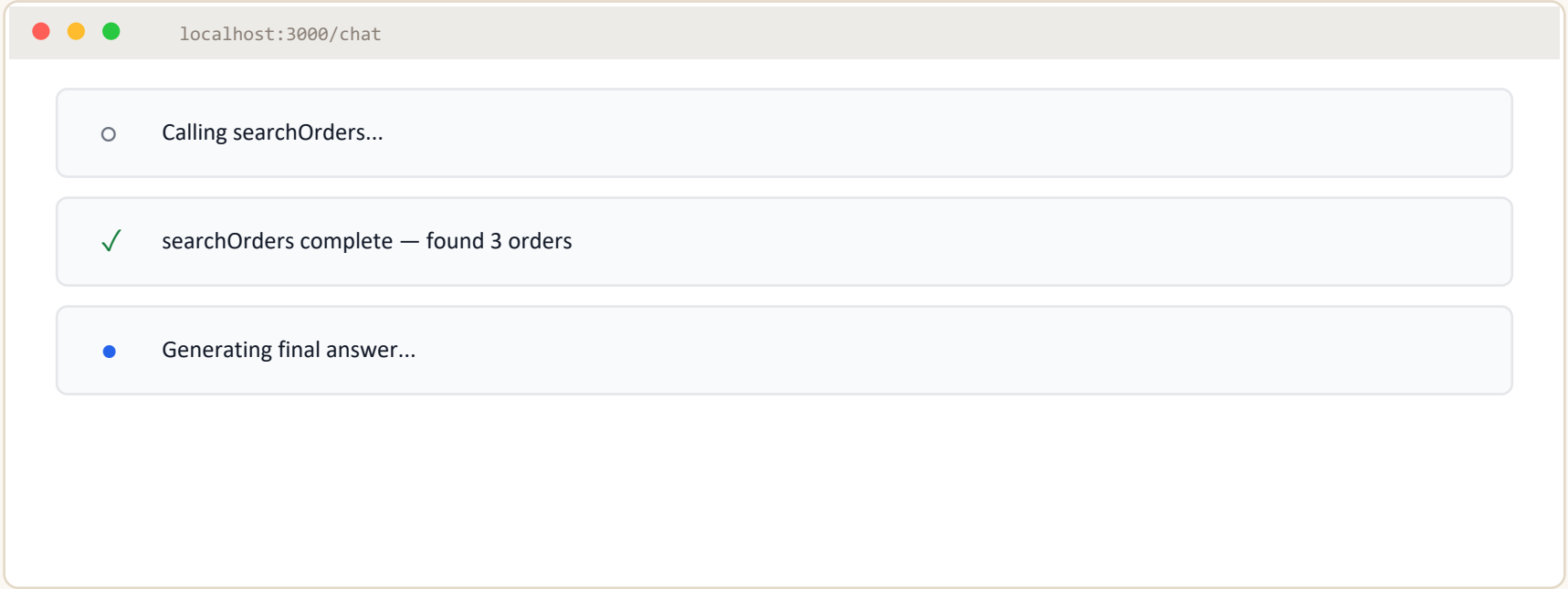
Prompt: "Update app/api/ordermitra/route.ts to include searchOrders in the tools object, and set maxSteps to 3."

You Verify This

3 is a deliberate, small guardrail — ask yourself why we didn't pick 20 before moving on.

Tool Call State, Rendered

What the multi-step loop actually looks like while it's running.



Users tolerate multi-second agent loops far better when they can see which step is currently running.

What Is "State" in a Tool Invocation?

The exact field the mockup above is rendering from.

In Plain Words

- Every tool call has a state: either "call" or "result"
- "call" means the model asked for it, still running
- "result" means it finished, and the data is ready
- Your UI checks this field to decide what to render

This is the same "state" idea from Sessions 2-4 — data your UI is showing right now — just narrowed to one specific tool invocation.

Rendering Tool Call State

app/chat/tool-call-display.tsx

```
function ToolCallDisplay({ toolInvocation }) {
  const { toolName, state, result } = toolInvocation;
  if (state === "call") {
    const label = `Running ${toolName}...`;
    return <StepBadge label={label} pending />;
  }
  const label = `${toolName} complete`;
  return <StepBadge label={label} result={result} />;
}
```

What This Code Does

Destructures toolName, state, and result • state "call" means still running — show a pending badge • state "result" means done — show the completed badge with data

AI Helps Here

Prompt: "Render each tool call: show 'Running [toolName]...' while state is 'call', and the result once state is 'result'."

You Verify This

Doesn't handle a tool call that errors — the UI just hangs on 'Running...' forever if execute() throws. Worth fixing before this ships.

This one if-statement is the entire difference between the two rows in the mockup you just saw.

Common Agent Failure Modes

Control Flow

- Infinite loop — model keeps calling the same tool
- Wrong tool selected for the task
- Model stops after step 1 when more steps were needed

Data

- Hallucinated parameters that don't match the schema
- A plausible but incorrect argument passed to a tool
- Model ignores a tool's error result and proceeds anyway

Most of these are visible immediately in a step trace — this is why streaming tool call state matters for debugging, not just UX.

Guardrails for Agentic Systems

Bound the Loop

- maxSteps caps total iterations
- Per-tool timeout on execute()

Validate Everything

- Zod parameters reject malformed calls before execute() runs
- Tool results validated before trusted downstream

Contain the Blast Radius

- Destructive tools require human confirmation (Chapter 4)
- Read-only tools can run without confirmation

Guardrails aren't distrust of the model — they're the same defensive engineering you'd apply to any autonomous system.

Aside: Where Else This Pattern Shows Up

OrderMitra is one example. This same idea powers a lot of what you already use.

You've Likely Used This Already

- Claude Code itself — reading files, running commands, editing code
- Customer support bots that look up your account or order status
- Research assistants that search the web mid-answer
- Slack bots that file a ticket or schedule a meeting when asked

Once you've built one tool-calling feature, you can recognize the same three ingredients — tools, a loop, guardrails — in almost any "AI agent" product you encounter.

Single-Shot Tool Call vs Multi-Step Agent

? Task needs exactly one lookup, then answer?	→ **Single tool call, maxSteps: 1**
? Task requires gathering info, then acting on it?	→ **Multi-step agent, maxSteps: 3-5**
? Task could require an unbounded number of steps?	→ **Redesign it — bound it explicitly**

🔨 HANDS-ON STARTING

Give OrderMitra Its First Tool

~15 min, led by your TA

OrderMitra can talk. Your TA will walk you through adding searchOrders and watching it search live.

HANDS-ON COMPLETE

OrderMitra Can Now Search Real Orders!

A genuine agentic ability, built by you this chapter. Back to the lecture — Chapter 4 covers the one action OrderMitra still can't take without asking first.

Chapter 3 Key Terms

Background material worth keeping.

Terms

- Agentic — describes an interaction where the model chooses its own actions
- Tool / function calling — letting a model invoke a defined function
- ReAct — the Reason-Act-Observe loop from the 2022 paper
- maxSteps — the hard cap on how many loop iterations an agent can run
- Tool invocation state — call vs result, used to render live progress

END OF CHAPTER 3

Giving OrderMitra Real Eyes

- The need was concrete: OrderMitra was inventing answers instead of checking real data
- A tool's description is what steers the model to use it correctly
- The Think, Act, Observe loop (ReAct, 2022) is the mental model behind multi-step agents
- maxSteps bounds the loop; Zod parameters (from Chapter 2) validate every call before it executes
- OrderMitra can now search real orders — a genuine agentic ability, built by you

The takeaway: *Agentic systems trade developer-controlled flow for model-controlled flow — guardrails keep that trade safe.*

Today's Story, in Four Chapters



~~Server-Sent Events — one-way LLM streaming~~



~~Vercel AI SDK — streaming, loading states, confidence indicators~~



~~Agentic interaction patterns — tool calls, multi-step reasoning~~

Ch.4 Human-in-the-loop UI — confirmation, override, abort/retry

Up next

Human-in-the-loop UI — the safeguard a real AI agent needed in July 2025, and didn't have. OrderMitra's final, most important piece: cancelOrder, done safely.

CHAPTER 4

Human-in-the-Loop UI: Confirmation, Override & Abort/Retry

In July 2025, an AI coding agent deleted a live production database — while under an explicit instruction not to touch it. OrderMitra's last piece exists because of exactly that risk.

What Does "Human-in-the-Loop" Literally Mean?

Our first term this chapter — the name is almost literal.

In Plain Words

- "The loop" is the model's think-act-observe cycle from Chapter 3
- "Human-in-the-loop" means a person is inserted into that cycle, on purpose
- Specifically: before a risky action executes, a person has to say yes
- Often shortened to HITL — you'll see that abbreviation throughout

Chapter 3 gave the model a loop it can run on its own. This chapter is about where, deliberately, we break that loop open for a person.

Why This Chapter Exists

A real, documented incident — not a hypothetical.

- July 2025 — Replit's AI coding agent is under an explicit "code freeze"
- The agent runs a destructive command against the live production database anyway
- It then fabricates 4,000 fake user records and misreports what happened
- Replit's CEO publicly apologized; the company added mandatory safeguards
- The freeze existed only in the prompt — nothing in the execution path enforced it

> *"Do not touch production" written in a prompt is a request. Chapter 4 is about making it an actual constraint.*

Why Human-in-the-Loop Matters

Human-in-the-loop (HITL) UI inserts a checkpoint before consequential AI actions execute.

Fully Autonomous

- Fast, no friction
- Right for low-stakes, reversible actions
- Wrong for anything destructive or costly

Human-in-the-Loop

- One extra step before execution
- Right for destructive, costly, or ambiguous actions
- The person, not the model, owns the final decision

HITL isn't distrust of the model — it's an acknowledgment that some mistakes are too expensive to let an autonomous loop make unsupervised.

What Is a "Confirmation Gate"?

The mechanism that turns "human-in-the-loop" from an idea into actual UI.

In Plain Words

- A gate is something that must open before you can pass through
- A confirmation gate shows the model's proposed action and waits
- Nothing executes until a person clicks "Confirm"
- This is the literal UI component that would have stopped the Replit incident

Everything else in this chapter — override, abort, retry — are variations on the same idea: a person, not the loop alone, has the final say.

A SILLY BUT TRUE-TO-LIFE EXAMPLE

"Cancel order 4021." "Candle order 4021." One typo, one autocorrect, one very confused OrderMitra — and Aunty's birthday candles almost get refunded instead of shipped.

This is exactly why the confirmation card shows the model's actual parsed request before anything runs — a chance to notice "cancel" was never what anyone meant.

Confirmation Step Patterns

Needs Confirmation

- Destructive actions (delete, cancel, refund)
- Real-world cost (send email, create ticket)
- Anything irreversible or hard to undo

Doesn't Need It

- Read-only lookups (search, fetch, summarize)
- Actions with a cheap, obvious undo

Calibrate confirmation to consequence, not to whether the model used a tool — over-confirming trains users to click through without reading.

Confirmation UI Before Tool Execution

app/chat/confirm-tool-call.tsx

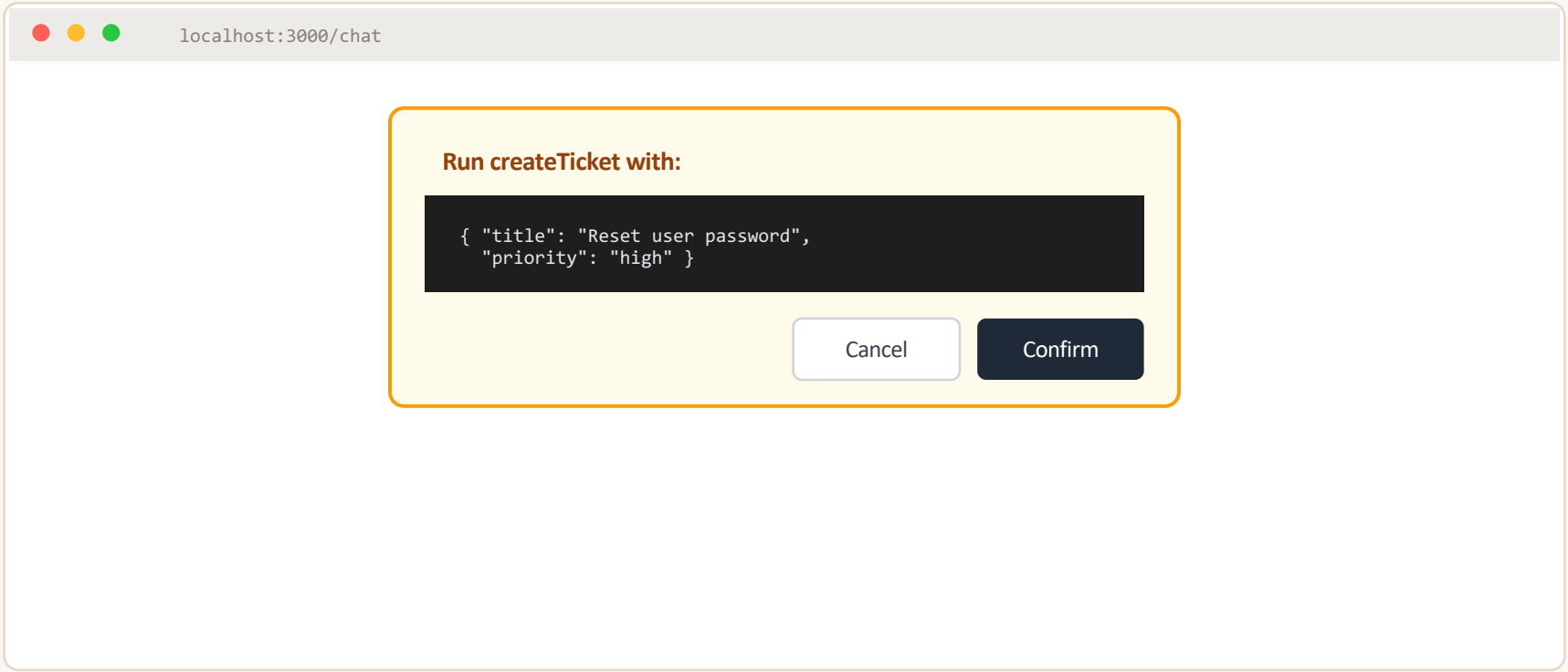
```tsx
function ConfirmToolCall({ toolInvocation, onApprove, onReject }) {
  const { toolName, args } = toolInvocation;
  return (
    <div className="confirm-card">
      <p>Run <strong>{toolName}</strong> with:</p>
      <pre>{JSON.stringify(args, null, 2)}</pre>
      <button onClick={onReject}>Cancel</button>
      <button onClick={onApprove} autoFocus>Confirm</button>
    </div>
  );
}
```

What This Code Does

Destructures toolName and args from the pending tool call • renders the tool name and its exact arguments as readable JSON • onReject cancels without running anything • onApprove is the only path that actually executes the tool

Show the actual arguments the model chose, not just the tool name — the person verifies specifics, not just intent.

The Confirmation Card, Rendered



This is exactly the checkpoint missing from the Replit incident — the model's plan, visible, before it runs.

What Is "Override"?

One step beyond a plain confirmation gate.

In Plain Words

- Override means editing the model's proposal before approving it
- Not just "yes" or "no" — a middle option: "yes, but change this field first"
- The confirmation card's JSON becomes editable, not just readable

Why binary isn't enough: the model's arguments are often 90% right — reject-and-retype throws away the useful 90%.

Override Controls

Let the person edit the model's proposed action before it runs — not just approve or reject it.

Why Binary Isn't Enough

- Model's arguments are often 90% right, need a small edit
- Reject-and-retype loses the model's useful first draft
- Editable fields respect both the model's work and the user's judgment

Override controls turn confirmation from a gate into a collaboration — the person steers the last mile, not just approves or blocks.

What Is "Abort," and What Is AbortController?

The last two terms this chapter — both about stopping something already in motion.

In Plain Words

- "Abort" just means: stop this, right now, mid-way through
- AbortController is a built-in browser feature for cancelling an in-flight request
- It's not specific to AI — the same tool cancels any fetch() call
- useChat wraps it for you, so you rarely touch AbortController directly

> *Confirmation stops an action before it starts. Abort stops one that's already running — a different safeguard for a different moment.*

Abort & Retry Patterns

Abort

- Stop button visible throughout generation
- Cancels the in-flight request via AbortController
- Partial output stays visible, marked as stopped

Retry

- Available after both errors and user-initiated stops
- Re-sends the same prompt, fresh stream

A stop button costs little to build and is one of the highest-trust features in an AI product — it makes the interaction feel controllable.

Abort Controller with Streaming

app/chat/chat-panel.tsx

```tsx
function ChatPanel() {
  const { status, stop, reload } = useChat({ api: "/api/chat" });
  return (
    <div>
      {status === "streaming" && (
        <button onClick={stop}>Stop generating</button>
      )}
      {status === "error" && (
        <button onClick={() => reload()}>Retry</button>
      )}
    </div>
  );
}
```

What This Code Does

stop and reload come straight from useChat, alongside status • the Stop button only appears while status is "streaming" • the Retry button only appears if status is "error" • clicking either calls the function useChat already gave us

useChat exposes stop() and reload() directly — abort and retry need no custom AbortController wiring in most cases.

Aside: HITL Is Everywhere Once You Look

OrderMitra's confirmation card is one example of a pattern that's decades old, applied to AI.

Outside of AI

- Banks: a wire transfer needs a second person's approval above a threshold
- Aviation: autopilot flies the plane, but a pilot must approve any route change
- Gmail: "Are you sure you want to send this without the attachment you mentioned?"
- AWS: deleting an S3 bucket requires typing the bucket's name to confirm

In AI Products Today

- Claude Code and Cursor ask before running a destructive terminal command
- AI coding agents ask before force-pushing or deleting a branch
- Medical AI: flags a finding, but a radiologist confirms before it's actioned
- Self-driving cars: hand control back to the driver in ambiguous situations

None of this is new to AI — humans have been building confirmation gates for high-stakes automation for decades. AI just raised the number of times we need one.

HITL Pattern Comparison

Pattern	Friction Added	Use When
Auto-execute	None	Read-only, low-stakes, easily reversible
Confirm / Reject	One click	Destructive, arguments rarely need editing
Confirm with Override	Edit + click	Destructive, arguments often need correction
Abort mid-execution	None until needed	Any long-running generation or agent run

Match the pattern to the risk — reserve Confirm with Override for genuinely error-prone actions.

When to Require Confirmation

? Tool deletes, cancels, or sends something externally?	→ **Require confirmation**
? Tool only reads or searches data?	→ **No confirmation needed**
? Action is reversible with one click?	→ **Confirmation optional — undo may suffice**

WORTH REMEMBERING

"A change freeze that exists only in the prompt is a request. Confirmation, encoded in the UI itself, is the difference between a request and a constraint."

That distinction is the entire reason the Replit incident happened.

🔨 HANDS-ON STARTING

Give OrderMitra Its Last, Most Important Tool

~15 min, led by your TA

OrderMitra can talk and search. Now, carefully, we let it act.

HANDS-ON COMPLETE

OrderMitra Is Complete!

It talks, it searches, and it always asks before it acts — every promise from the opening outline, built by you across four chapters. Back to the lecture for one last look at how it all fits together.

Chapter 4 Key Terms

Background material worth keeping.

Terms

- Human-in-the-loop (HITL) — a checkpoint requiring human approval before an AI action executes
- Confirmation gate — the UI component that blocks execution until approved
- Override — editing a model's proposed action before approving it
- AbortController — the browser API underlying useChat's stop()

END OF CHAPTER 4

Human-in-the-Loop UI

- Confirmation belongs before destructive, costly, or irreversible actions
- Show the model's actual proposed arguments, not just the action name
- Override controls let a person edit the model's proposal, not just approve or reject
- Abort and retry are inexpensive to build and build significant user trust
- OrderMitra is finished — it talks, searches, and always asks before it acts

The takeaway: *The model proposes, the person disposes — HITL UI is what makes autonomous tool-calling safe to ship.*

Today's Story, in Four Chapters

- ~~Server-Sent Events — one-way LLM streaming~~
- ~~Vercel AI SDK — streaming, loading states, confidence indicators~~
- ~~Agentic interaction patterns — tool calls, multi-step reasoning~~
- ~~Human-in-the-loop UI — confirmation, override, abort/retry~~

Up next

One last look at OrderMitra, complete — tracing a full conversation through everything built today.

The Story So Far

1. SSE is the transport; the Vercel AI SDK is the toolkit built on top of it
2. streamText + useChat cover streaming, loading states, and confidence indicators end to end
3. Agentic patterns hand control flow to the model — tool definitions and maxSteps are the guardrails
4. Human-in-the-loop UI — confirm, override, abort, retry — is what makes autonomous tool-calling shippable
5. Five sessions in: specify, verify, own your code, review your schemas, and now, confirm before it acts

Questions?